WEEK 6 ASSIGNMENT: SPARK ARCHITECTURE, TRANSFORMATIONS, & PERFORMANCE

Q1: Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

- Driver Program: The master process that runs your main application code. It is responsible for creating the SparkSession, converting your code into a logical plan (DAG), scheduling tasks, and coordinating with the executors.
- Cluster Manager: An external service (like standalone Spark, YARN, Kubernetes, or Mesos) that monitors resources across the cluster. Its sole job is to allocate hardware resources (CPU cores and memory) to the Spark application when requested by the Driver.
- Executors: Worker processes launched on cluster nodes. They are responsible for storing data in memory/disk and executing the individual processing tasks assigned to them by the Driver. Once tasks finish, they return results to the Driver.

Q2: How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

- Lazy evaluation means Spark does not execute transformations (e.g., filter(), select()) immediately when they are written. Instead, it records them as a lineage graph (DAG).

- This improves performance because Spark's Catalyst Optimizer looks at the entire chain of transformations right before an Action is called. It can optimize execution by combining steps (like merging multiple filters together) or skipping unnecessary data reads, avoiding redundant memory writes and cluster overhead.

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

- ```
  df = (spark.read
  .option("header", "true")
  .option("inferSchema", "true")
  .csv("data/source.csv"))
  ```
  ```

Q4: What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

- CSV (Row-Based): Stores data line-by-line. To read a single column (e.g., just price), Spark must scan every single row and every single column on disk. This causes heavy disk I/O bottlenecks.
- Parquet (Columnar): Groups data by columns rather than rows. If you only need 2 columns out of 100, Spark only reads those 2 columns off the disk. Furthermore, columnar data is highly compressible (since data types are identical per column), which vastly reduces storage footprints and speeds up execution times.

Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

- from pyspark.sql.functions import col
  result_df = df.filter(col("category") == "Electronics").select("product_id", "price")

Q6: Write the code to "revise" a DataFrame by renaming the column old_name to new_name and casting the price column from a String to a Double.

- from pyspark.sql.functions import col
  revised_df = (df
  .withColumnRenamed("old_name", "new_name")
  .withColumn("price", col("price").cast("double")))

Q7: How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

- Because Spark records every transformation applied to a dataset as an immutable Lineage Graph (DAG), it knows the exact step-by-step history of how any given data partition was constructed. If a worker node crashes and a portion of data is lost, Spark does not restart the entire job. It simply re-allocates the tasks for the missing data partition to another healthy node and rebuilds *only* that missing piece using the lineage instructions.

Q8: Write a query to filter a DataFrame df_orders for rows where the status is 'Completed' AND the amount is greater than 1000.

- from pyspark.sql.functions import col
  filtered_orders = df_orders.filter((col("status") == "Completed") & (col("amount") > 1000))

Q9: Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.

- Parquet files contain metadata blocks that store statistics (like minimum and maximum values) for each column chunk. Predicate Pushdown is an optimization where Spark pushes filtering logic (e.g., age > 21) directly down to the storage layer *before* reading the files.
- If the metadata shows a file block only contains ages between 10 and 18, Spark skips reading that block entirely. This drastically slashes disk I/O and prevents irrelevant data from wasting RAM.

Q10: Write a code snippet to add a new column final_price which is the base_price multiplied by 1.18 (18% tax).

- from pyspark.sql.functions import col
  df_with_tax = df.withColumn("final_price", col("base_price") * 1.18)

Q11: What is the difference between Transformations and Actions? Provide two examples of each.

- Transformations: Operations that define how to modify a DataFrame. They are lazy and merely build the execution plan without executing it.- Examples: .filter(), .select(), .withColumn(), .groupBy()
- Actions: Operations that instruct Spark to execute the recorded transformations and compute the final output. They trigger actual computation across the cluster.- Examples: .show(), .count(), .write, .collect()

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

- ```
  from pyspark.sql.functions import col
  # Load
  df_parquet = spark.read.parquet("path/to/input")
  # Filter nulls
  df_filtered = df_parquet.filter(col("user_id").isNotNull())

  # Save as CSV
  (df_filtered.write
  .mode("overwrite")
  .option("header", "true")
  .csv("path/to/output"))
  ```

Q13: In Spark Architecture, what is the difference between Client Mode and Cluster Mode?

- Client Mode: The Driver program runs directly on the machine where the user submitted the job (e.g., your laptop or a gateway edge node). If you close your terminal or disconnect from the network, the job dies. It is typically used for interactive testing and debugging.
- Cluster Mode: The Driver program is offloaded and runs inside a worker node (an executor container) somewhere deep inside the cluster. You can safely disconnect your local machine; the job will continue executing automatically until it finishes. This is used for production deployments.

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

- ```
  from pyspark.sql.functions import col
  result_df = df.filter((col("region") == "North") | (col("priority") == "High"))
  ```

Q15: When exploring a dataset, why is it safer to use .show(5) instead of .collect() on a multi-terabyte dataset?

- .show(5) is a conservative operation. It fetches only the first 5 records from the nearest data partitions and pulls them to your console, consuming almost no overhead.
- .collect() tells Spark to pull every single row across the entire multi-terabyte cluster and aggregate them into the single JVM memory space of your Driver program.
- ➢ Because a standard driver machine doesn't have terabytes of RAM, calling .collect() on large datasets will cause an immediate OutOfMemory (OOM) Error and crash your entire Spark application.